

PROgramming in LOGic

CS 331 Tutorial # 1

Rashmi Dutta Baruah

Department of Computer Science & Engineering
IIT Guwahati

Outline

- Introduction
- Prolog Syntax & Semantics
 - Data objects: simple objects & structures
 - Simple objects: constants & variables
 - Clauses: facts, rules, questions
 - Matching operation
 - Program meaning: declarative and procedural meaning
 - Rules revisited- recursive rules

% Facts

```
parent(john, mary).  
parent(john, tom).  
parent(susan, mary).  
parent(susan, tom).
```

```
male(john).  
male(tom).  
female(susan).  
female(mary).
```

% Rules

```
father(X, Y) :-  
    parent(X, Y),  
    male(X).
```

```
mother(X, Y) :-  
    parent(X, Y),  
    female(X).
```

```
sibling(X, Y) :-  
    parent(P, X),  
    parent(P, Y),  
    X \= Y.
```

Queries

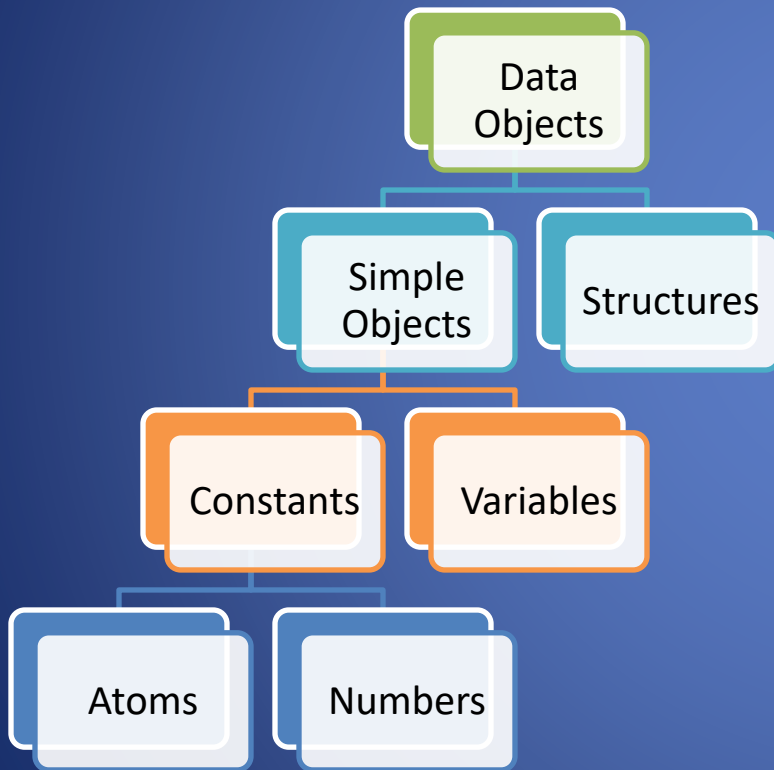
```
?- father(john, mary).  
true.
```

```
?- mother(susan, tom).  
true.
```

```
?- sibling(mary, tom).  
true.
```

```
?- father(X, mary).  
X = john.
```

Prolog Syntax and Semantics



Atom: starts with a lower case letter

- strings of letters, digits and underscore character e.g. `alpha_5`
- Strings of special characters e.g. `<====>`
'`:-`' reserved
- Strings of characters enclosed in single quotes, useful for atoms that starts with capital letter e.g. `'Tom_Jerry'`

Variable: starts with an upper case letter or an underscore

- Strings of letters, digits, and underscore e.g. `Student_list1`

Prolog Syntax and Semantics

- 'anonymous' variable written as single underscore character (will be discussed later).
- Scope of a variable is one clause.

Structures: objects with several components

- components can be structures themselves
e.g. `date(12, sept, 2014)`
- `date` – functor and
- `12, sept...` - arguments

All data objects in Prolog are TERMS.

Prolog Syntax and Semantics

clauses

- facts, rules, questions; consist of **head** and **body**.
- body: list of goals separated by commas (conjunctions) .
- `mother(X, Y) :- parent(X, Y), female(X) .`

facts

- declare things that are unconditionally true.
- clauses without a body.
- `female('Jerry') .`

rules

- declare things that are true depending on a given condition.
- clauses with head and (non-empty) body.
- `mother(X, Y) :- parent(X, Y), female(X) .`

questions

- ask the program what things are true.
- clauses with only body.
- `?- mother('Jerry', 'Sam') .`

Prolog Syntax and Semantics

- Which of the following rules are syntactically correct?

```
likes(X,Y) :- a,b.
```

Yes.

```
eats(food) :- blue(blueberry), red(strawberry).
```

Yes.

```
f(a) :- f(b), f(c) :- f(d) f(e).
```

No.

```
happy(X) :- hasmoney(X) & hascar(X).
```

No.

Prolog Syntax and Semantics

Matching : The process of matching is called Unification.

- Two terms S and T match
 1. If S and T are constants then S and T match only if they are the same object.
 2. If S is a variable and T is anything, then they match, and S is instantiated to T. Conversely, if T is a variable then T is instantiated to S.
 3. If S and T are structures then they match only if
 - S and T have the same principal functor, and
 - all their corresponding components match.The resulting instantiation is determined by the matching of the components.

Prolog Syntax and Semantics

The process of matching the head of a clause with goal to be solved is the key to execution of logic programs.

- An example database that details 'who' likes 'what' movies in some world model.

```
likes(gaurav, action).  
likes(ajay, action).  
likes(rahul, comedy).  
likes(nisha, romcom).
```

- Now, some simple questions. Answer 'yes' if there is a match (unification successful) otherwise 'no'.

Prolog Syntax and Semantics

- Now, some simple questions. Answer 'yes' if there is a match (unification successful) otherwise 'no'.

```
likes(gaurav, action).  
likes(ajay, action).  
likes(rahul, comedy).  
likes(nisha, romcom).
```

```
?-likes(akshay, action). /* does akshay likes action movies? */  
no  
?-likes(ajay, what). /* what does ajay like? */  
no  
?-likes(rahul, What). /*what does rahul like? */  
What = comedy          /* What matches (unifies) with comedy */  
yes
```

Prolog Syntax and Semantics

- Some more on matching

```
eats(ajay, sandwich).  
eats(Who, What).  
Yes  
Who = ajay  
What = sandwich
```

```
eats(ajay, Food).  
eats(What, sachin).  
Yes  
What = ajay  
Food = sachin
```

```
f(X, Y).  
f(A, B).  
Yes  
X = A  
Y = B
```

```
f(a, B).  
f(X, Y).  
Yes  
X = a  
B = Y
```

Prolog Syntax and Semantics

- Two levels of meaning of Prolog programs:

```
mother(X, Y) :- parent(X, Y), female(X).
```

Declarative meaning

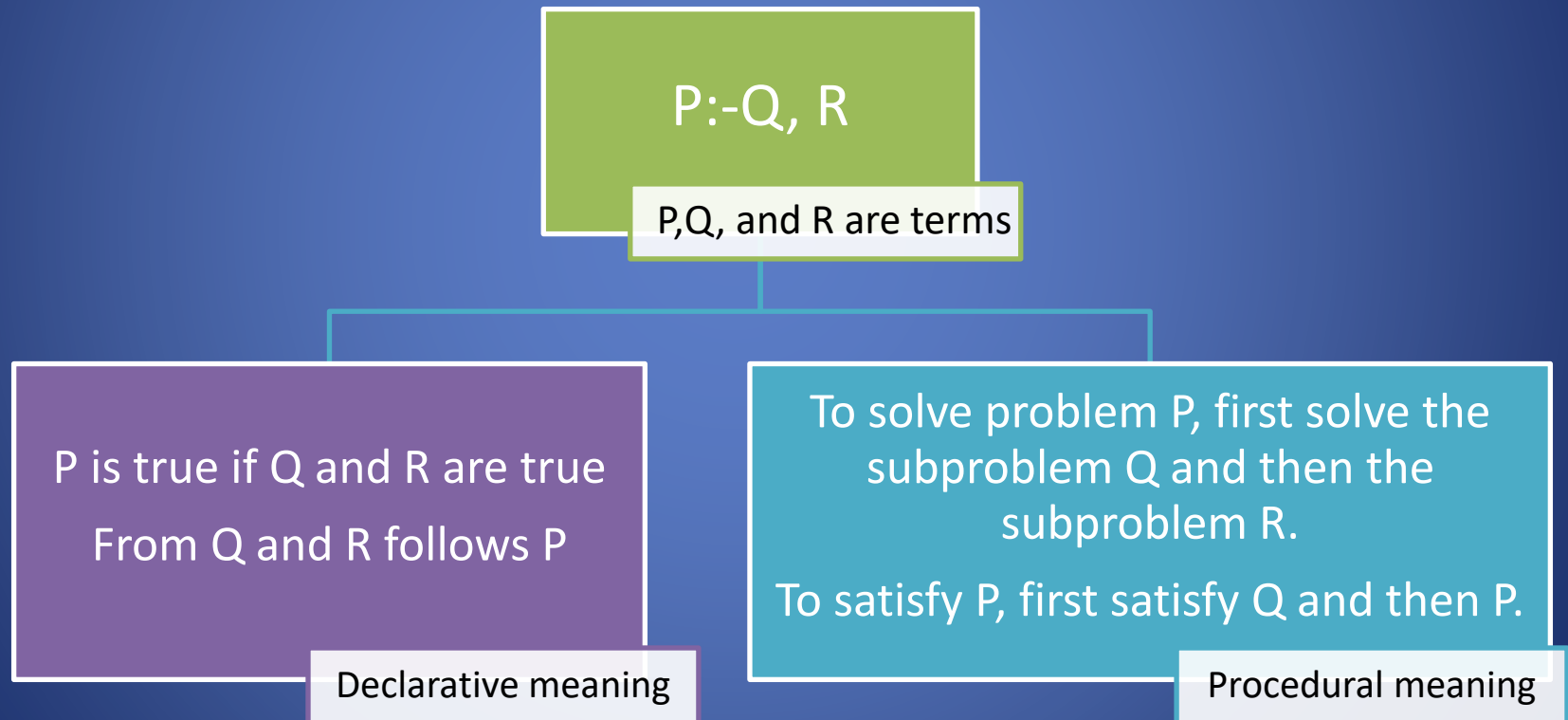
- determines 'what' will be the output of the program.
- For all X and Y, X is mother of Y if X is a parent of Y and X is female

Procedural meaning

- determines 'how' the output will be obtained and the relations will be evaluated by Prolog system.
- to prove the goal, X is mother of Y first prove the subgoal X is parent of Y and then the subgoal X is female.

Prolog Syntax and Semantics

- In general,



Prolog Syntax and Semantics

- Declarative meaning
 - Instance of a clause C : each variable substituted by some term.
 - Variant of a clause C : instance where each variable is substituted by another variable.

```
mother(X, Y) :- parent(X, Y), female(X).
```

Instance:

```
mother(jane, A) :- parent(jane, A), female(jane).
```

Variant:

```
mother(A, B) :- parent(A, B), female(A).
```

Prolog Syntax and Semantics

Given a program and a goal G , the declarative meaning says:

A goal G is true (that is, satisfiable, or logically follows from the program) if and only if

1. there is a clause C in the program such that
2. there is a clause instance I of C such that
 - (a) the head of I is identical to G , and
 - (b) all the goals in the body of I are true

Prolog Syntax and Semantics

- Procedural meaning
 - Procedure for executing a list of goals given a program.
 - Input: a program and a goal list.
 - Output: a success/failure indicator and instantiation of variables
 - Success/failure indicator: 'yes' if the goals are satisfiable and 'no' otherwise
 - Instantiation of variables is produced only in case of successful termination.

Prolog Syntax and Semantics

- Procedural meaning continues...

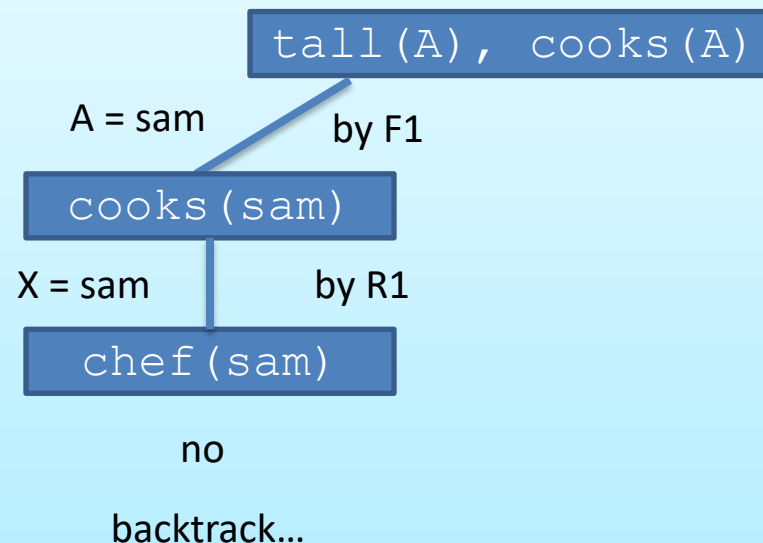
PROGRAM/KB/DB

```
tall(sam).           %F1
tall(joe).           %F2
average(tom).        %F3
musician(sam).       %F4
chef(joe).           %F5
sportsman(tom).      %F6
cooks(X):- chef(X).  %R1
plays(X):-
    sportsman(X).    %R2
plays(X):-
    musician(X).     %R3
```

QUESTION

```
?- tall(A), cooks(A).
/* who is tall and knows
how to cook */
```

EXECUTION TRACE



Prolog Syntax and Semantics

- Procedural meaning continues...

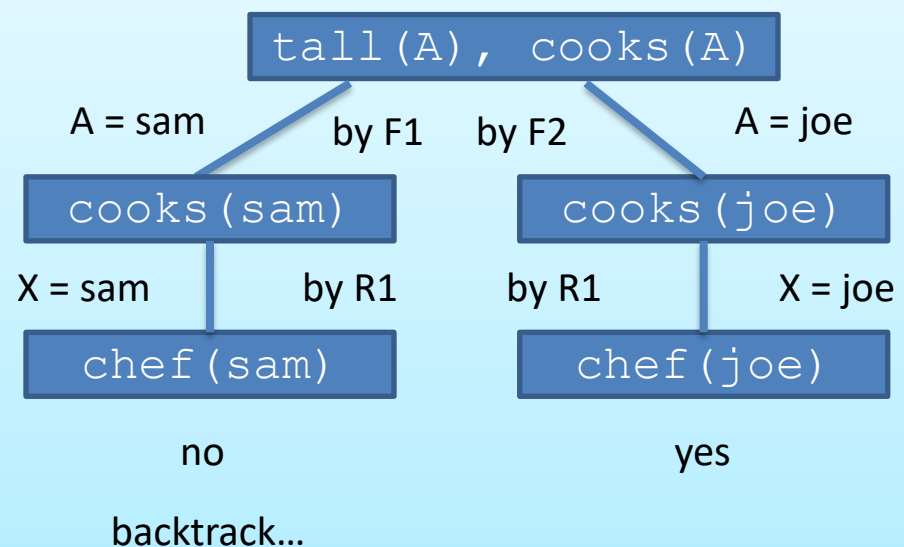
PROGRAM

```
tall(sam).           %F1
tall(joe).           %F2
average(tom).        %F3
musician(sam).       %F4
chef(joe).           %F5
sportsman(tom).      %F6
cooks(X):- chef(X).  %R1
plays(X):-
    sportsman(X).    %R2
plays(X):-
    musician(X).     %R3
```

QUESTION

```
?- tall(A), cooks(A).
/* who is tall and knows
how to cook */
```

EXECUTION TRACE



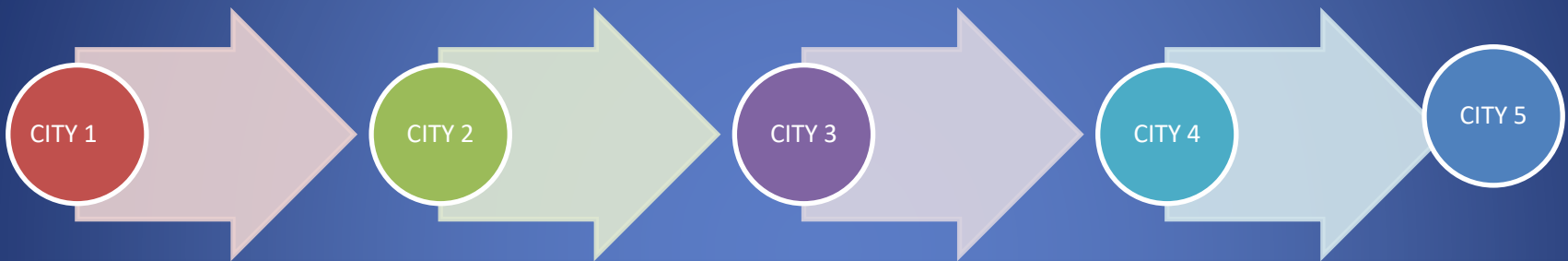
Prolog Syntax and Semantics

- Procedural meaning continues...

```
WHILE (the goal list is non-empty)
    select the first goal in the list.
    scan the program from top to bottom looking for a clause whose head
    matches the goal.
    IF (such a clause is found)
    THEN
        replace the goal by the body of the matching clause .
        Instantiation of variables due to the matching are applied to
        the entire goal list.
    ELSE
        restore the goal list and bindings as they were prior to the
        most recent clause selection (i.e. backtrack)
        If there is no previous state to which we can backtrack i.e.
        current goal list is the initial goal list then terminate with
        FAILURE.
END WHILE
SUCCESS
```

Prolog Syntax and Semantics

- Rules revisited – recursive rule
 - Rule referring to itself



A one way road links five cities. The task is to determine if one can get from one city to another.

```
can_get(City1,City5):-  
link(City1,City5).  
can_get(City1,City5):-  
link(City1,City2),link(City2,City5).  
can_get(City1,City5):-  
link(City1,City2),link(City2,City3),  
link(City3,City5).  
...
```

```
can_get(City1,City5):-  
link(City1,City5).  
can_get(City1,City5):-  
link(City1,X),can_get(X,City5).
```

Recursive rule

Prolog Syntax and Semantics

- Consider the following program. Write down all the variable bindings that the variable *X* gets when the given query is executed.

```
f(X) :- a(X), b(X).  
f(X) :- c(X), b(X).  
a(b).  
a(c).  
a(3).  
a(2).  
b(5).  
c(1).  
  
?-f(X).
```

The bindings for *X* are b, c, 3, 2, 1.

Getting started

- Installing SWI-Prolog (Ubuntu)
 - Type the following at the prompt.

```
% sudo apt-get install swi-prolog
```

- You can create Prolog files using your system's basic text editor and saving it with extension ".pl".
- Start Prolog by typing "swipl" at the prompt.
- Consult (load) Prolog file by typing "[filename]." at the Prolog prompt (full stop at the end is significant).
 - You can then enter any query against the predicates defined in the file.
- Quit Prolog by typing "halt."

Summary

- What we discussed today?
 - atoms, variables, structures
 - clauses: facts, rules (also recursive rules), and questions.
 - Matching process c(unification).
 - Declarative and procedural meaning.